Palm OS Programming Recipes

Making a Speech Telephone Call with the Telephony Manager

Because phone manufacturers interpret standards differently, coding a telephony application can be difficult and often needs a complex database of the specifics for each model or brand. Introduced in Palm OS® software version 4.0, the Telephony Manager relieves developers of this burden; an application using the Telephony Manager APIs can be sure to work with any phone, if the driver for the phone has been installed.

In the following major steps, this recipe shows you how to make a speech telephone call from a Palm OS application using the Telephony Manager:

1. Initialize.
2. Check whether the telephony feature is available.
3. Call a telephone number.
4. Exit and clean up.

At the end, you will find some tips for extending the basic functionality presented below. **1. Initialize**

As usual, the first thing to do is to include the appropriate header files in your code:

```
#include <TelephonyMgr.h>
#include <TelephonyMgrTypes.h>
```

Before using the telephony functions, you need to load the Telephony Manager Library, and then to open it to get the application attachment identifier needed by the APIs. The steps shown in the following code are:

1. Check whether the library is already loaded.
2. If it's not, load it.
3. Open the loaded library.

```
UInt16 gTelRefNum;
TelAppID gTelAppID;
Err err;

err = SysLibFind(kTelMgrLibName, &gTelRefNum);
if (err !=0)
{
   /* The library is not loaded, so load it. */
   err = SysLibLoad(kTelMgrDatabaseType,
      kTelMgrDatabaseCreator, &gTelRefNum);
   if (err)
      /* Problem... Display a message. */
}

err = TelOpen(gTelRefNum, kTelMgrVersion, &gTelAppID);
/* Check err here. */
```

You're now ready to make a phone call.

2. Check whether the Telephony Feature is Available

Important: As a rule, don't take for granted that a telephony feature is allowed by your phone driver. Use the `TelIs<Service>Available` and `TelIs<function>Supported` macros. For example, data calls are often not implemented.

In the case of a speech call, it should not be an issue: both `TelIsSpcServiceAvailable (gTelRefNum, gTelAppID, NULL)` and `TelIsSpcCallNumberSupported (gTelRefNum, gTelAppID, NULL)` should return `errNone`.

But, for example, if you want to make a data call instead of a voice call, with `TelDtcCallNumber`, you must verify whether the function is supported by checking that `TelIsDtcCallNumberSupported(gTelRefNum, gTelAppID, NULL)` returns `errNone`. For user interface reasons, you may also want to check whether the complete service is available with `TelIsDtcServiceAvailable(gTelRefNum, gTelAppID, NULL)`.

3. Call a Telephone Number

This function initiates the call:

```
err = TelSpcCallNumber(gTelRefNum, gTelAppID, phoneNumber, NULL);
```

The `phoneNumber` variable is a string like "5554321" or "+16265554321,,*123#". Check the definition of the `TelSpcCallNumber` function in the *Palm OS Programmer's API Reference* for more details.

Calling a number is simple, but you will notice a problem immediately: your application is frozen until the call has succeeded or failed. This problem comes from the fact that `TelSpcCallNumber` has been used in a synchronous mode. The synchronous mode is sometimes useful, depending on your needs, but, in this case, the asynchronous mode is probably better.

You can choose the synchronous mode by setting the last parameter (the transaction ID pointer) of a telephony function to `NULL`. Or you can choose the asynchronous mode by setting it to an `Int16` pointer.

```
UInt16 gTransID;
err = TelSpcCallNumber(gTelRefNum, gTelAppID, phoneNumber, &gTransID);
```

This way, the function returns immediately, and `gTransID` identifies the transaction.

The result of the asynchrounous function call is sent as an event. So, in your main event loop, you must test the event type to see whether `event.eType == kTelTelephonyEvent`. If that's the case, you can decode the event as a `TelEventType`:

```
typedef struct _TelEventType
{
   eventsEnum eType;
   Boolean penDown;
   UInt8 tapCount;
```

```
   Int16 screenX;
   Int16 screenY;
   UInt16 functionId;
   UInt16 transId;
   MemPtr* paramP;
   Err returnCode;
} TelEventType;
```

An event corresponding to `TelSpcCallNumber` has the following characteristics:

- `functionId` is set to "k" + function name + "Message", so in this example it should be `kTelSpcCallNumberMessage`.
- The `transID` field will be equal to `gTransID`. This field is useful only if you make several asynchronous calls simultaneously and need to match calls with notifications.
- `returnCode` value is `errNone` if the function succeeded, or an error code if the function failed. Error codes are defined for the `TelSpcCallNumber` function in the *Palm OS Programmer's API Reference*. You can get one of these errors, or a more general one:
  - `telErrSpcCallError`: the voice telephone call encountered an error.
  - `telErrSpcLineIsBusy`: the voice telephone call failed.
  - `telErrSpcLineIsReleased`: the voice telephone call has been released.

Most of the time, you let the user close the line directly on the phone because:

- You don't know when you have to close the line -- it depends on the time the user spends on the phone.
- You probably don't want to keep the communication port open, consuming batteries, during the call.
- With an infrared connection to the phone, the link is broken when the user puts the phone beside his or her ear.

However, if you must hang up the phone call from your application, you can use `TelSpcCloseLine`:

```
err = TelSpcCloseLine(gTelRefNum, gTelAppID, 0, &gTransID);
```

4. Exit and Clean Up

When you have finished with the Telephony Manager Library, remember to close it and unload it, but unload it only if the library has been loaded by your application itself. Use something like this:

```
if (TelClose(gTelRefNum, gTelAppID) != telErrLibStillInUse)
   SysLibRemove(gTelRefNum);
```

Tips

To extend the basic functionality introduced in this recipe, keep the following tips in mind:

- If you intend to make several telephony calls in a row, you should call `TelOpenPhoneConnection` at the beginning and `TelClosePhoneConnection` at the end. This way, the communication port stays open between the calls and the initialization string is not sent each time, leading to better performance.

- **Use with caution.** To receive notifications such as `telErrSpcLineIsReleased` when the phone call is released, the communication port must be opened with `TelOpenPhoneConnection`. However, this operation consumes batteries and prevents the handheld from going into sleep mode.
- To get the current state of the phone call, use `TelGetCallState`. You can get information such as "an outgoing voice call is being redialed", for example.
- **Be very careful with `TelEmcCall`**. On an actual handheld, this function really calls emergency services. For test purpose, use Palm OS Emulator. Instead of calling the actual emergency services number, Emulator calls the first entry in the current phone book.